Tiny C++ — Reference

Tiny C++ (tcpp) is a small C++ that is a **strict superset of C**: plain C compiles unchanged, and on top you get classes, single inheritance, virtual methods, constructors, references, and `new` / `delete`. It is built with `lex` + `yacc` + `cc` (`cpp.l` / `cpp.y` translate Tiny C++ → C, which `cc` lowers to a native .NET exe). libc is built in — no `#include` needed.

```
tcpp prog.cpp -o prog.exe       # compile to a native .NET exe
prog.exe                        # run it
```

Quick Reference

```
int n; double x; char c;            // C scalars; printf is built in
int a[5];  int* p = &a[0];          // arrays, pointers
if (c) { } else { }   while (c) { }   for (i = 0; i < n; i = i + 1) { }
class Box {                         // a class = struct + methods + vtable
    int v;                          // fields (private by default)
  public:
    Box(int x) { v = x; }           // constructor
    int get() { return v; }         // method (this-> implicit)
    virtual int area() { return 0; }// virtual -> dynamic dispatch
};
class Square : public Box { ... };  // single inheritance
Box  b;     Box* h = new Square();  delete h;     // stack / heap objects
void swap(int& a, int& b) { ... }   // reference parameters
```

Data types

C scalars `int` (Int32), `char`, `double` / `float` (Double), plus pointers, arrays, `struct`, and **classes**. A class is laid out as a struct with a hidden vtable pointer at offset 0, so a derived-class pointer is layout-compatible with its base (the basis for polymorphism). Strings are C strings (`const char*`); string literals work with `printf("%s", …)`.

Statements / Commands

The C statement set: declarations, assignment, `if` / `else`, `while`, `for`, `return`, `break` / `continue`, blocks, expression statements. Plus C++ object statements: object declarations (which run constructors), method calls (`obj.m()` / `ptr->m()`), `new` / `delete`.

Functions

Free functions as in C (`int add(int a, int b) { return a + b; }`) and **methods** inside classes. Parameters may be by value or **by reference** (`int& x`). Functions are `public static` methods on the emitted type, so a compiled program is callable from C#/VB.NET. Constructors initialize objects (and inherited base constructors run).

Input / Output

Through built-in libc: `printf` with `%d %g %c %s` , `putchar` , etc. No `#include` .

Graphics

None. Activity 8 below is **text art** via `printf` .

Notes

- A strict **C superset**: existing C compiles as-is.
- Classes use a single-inheritance vtable: `virtual` methods dispatch dynamically through base-class pointers (stack or heap); non-virtual calls are static.
- `new` / `delete` map to `malloc` / `free` in a flat memory arena; there is no GC.
- `for` is C89-style — declare the loop variable first (`int i; for (i = 0; …)`).

Subset boundaries

"Tiny" — not full C++. Not included: templates, the STL, namespaces, operator overloading, multiple inheritance, exceptions, `references` to anything but scalars in some positions, RAII destructors on scope exit, `auto` /range-for, and `std::string` (use C strings). Header `#include` is unnecessary (libc is built in) and not processed.

Tutorial

Every example was compiled with `tcpp` and run; the output shown is real.

1. Your first program

```
int main() { printf("Hello, Tiny C++!\n"); return 0; }
```

```
Hello, Tiny C++!
```

`main` returns `int` ; `printf` is built in (no `#include`).

2. Variables and data types

```
int main() {
    int    n = 42;
    double pi = 3.14159;
    char   c = 'A';
    printf("%d %g %c\n", n, pi, c);
    return 0;
}
```

```
42 3.14159 A
```

3. Flow control

```
int main() {
    int sum = 0;
    int i;
    for (i = 1; i ≤ 5; i = i + 1) sum = sum + i;
    printf("sum 1..5 = %d\n", sum);
    if (sum > 10) printf("big\n"); else printf("small\n");
    return 0;
}
```

```
sum 1..5 = 15
big
```

`for`, `while`, and `if` / `else` are standard C. (Declare the loop variable before the `for` — C89 style.)

4. Arrays

```
int main() {
    int a[5];
    int i;
    for (i = 0; i < 5; i = i + 1) a[i] = i * i;
    printf("a[3] = %d\n", a[3]);
    return 0;
}
```

```
a[3] = 9
```

C arrays index from 0. Arrays of *pointers* are central to polymorphism — see Activity 5, where an `Animal* arr[3]` holds objects of different derived types.

5. Subroutines and functions

Classes, inheritance, and **virtual** methods — the heart of Tiny C++:

```
class Animal {
  public:
    int legs;
    Animal() { legs = 4; }
    virtual const char* sound() { return "..."; }
    void describe() { printf("%s has %d legs and says %s\n", kind(), this->legs, sound()); }
    virtual const char* kind() { return "animal"; }
};
class Dog : public Animal {
  public:
    virtual const char* sound() { return "woof"; }
    virtual const char* kind() { return "dog"; }
};
class Puppy : public Dog {
  public:
    virtual const char* sound() { return "yip"; }
};
int main() {
    Animal a; Dog d; Puppy p;
    a.describe(); d.describe(); p.describe();
    Animal* arr[3]; arr[0] = &a; arr[1] = &d; arr[2] = new Puppy();
    int i;
    for (i = 0; i < 3; i = i + 1) printf("  -> %s\n", arr[i]->sound());
    return 0;
}
```

```
animal has 4 legs and says ...
dog has 4 legs and says woof
dog has 4 legs and says yip
  -> ...
  -> woof
  -> yip
```

`describe()` calls the **virtual** `kind()` / `sound()` on `this`, so each animal reports its own kind; `Puppy` inherits `Dog`'s `kind` ("dog") but overrides `sound` ("yip"). The base constructor sets `legs` for every subclass. Through `Animal* arr[]`, dispatch is dynamic.

6. Memory management

Tiny C++ exposes the object/vtable model and manual heap:

- **Stack objects** (`Square sq;`) run their constructor and install the vtable automatically; they live for the enclosing block.
- **Heap objects** (`new Square()`) are `malloc`'d (with the vtable installed and the constructor run); `delete` frees them.
- A derived object is layout-compatible with its base (vtable pointer at offset 0), so a base pointer can address either and `virtual` calls go to the right override.

```
class Shape {
  public:
    virtual int area() { return 0; }
    virtual const char* name() { return "shape"; }
};
class Square : public Shape {
    int s;
  public:
    void setside(int x) { s = x; }
    virtual int area() { return s * s; }
    virtual const char* name() { return "square"; }
};
int main() {
    Square sq; sq.setside(4);
    Shape* p = &sq;                       // base pointer to a stack object
    printf("%s area=%d\n", p->name(), p->area());
    Shape* h = new Square();              // heap object
    printf("heap name=%s\n", h->name());
    delete h;
    return 0;
}
```

```
square area=16
heap name=square
```

7. Strings and a text layout

References let a function modify its caller's variables; combined with C strings you lay text out by hand:

```
void swap(int& a, int& b) { int t = a; a = b; b = t; }
int main() {
    int x = 1, y = 2;
    swap(x, y);
    printf("x=%d y=%d\n", x, y);
    return 0;
}
```

```
x=2 y=1
```

`int& a` is a reference parameter: `swap` exchanges the caller's `x` and `y` in place.

8. Drawing a picture

No graphics — **text art** with `printf`:

```
int main() {
    int r = 1;
    while (r ≤ 4) {
        int c = 0;
        while (c < r) { printf("*"); c = c + 1; }
        printf("\n");
        r = r + 1;
    }
    return 0;
}
```

```
*
**
***
****
```

9. Where to go next

Because plain C compiles unchanged and classes add only a vtable, Tiny C++ is a good lens on how C++ objects work under the hood. A compiled program's functions are `public static` methods, so C#/VB.NET can call them. Next: combine references with classes, build deeper inheritance hierarchies, and read the generated C to see the struct-plus-vtable lowering. Templates/STL/namespaces are the explicit non-goals (*Subset boundaries*).